

chamada recursiva 138
 chamadas 115
 chamadora, função 115
 classe de armazenamento 132
 classe de armazenamento de um identificador 132
 coerção de argumentos 122
 constante de enumeração 132
 corpo de função 119
 desempilhar 123
 deslocamento 125
 dividir e conquistar 114
 duração de armazenamento automático 133
 duração de armazenamento estático 133
 duração do armazenamento 132
 efeitos colaterais 124
 empilhar 123
 enum 132
 enumeração 132
 escala 125
 escopo 132
 escopo de arquivo 135
 escopo de bloco 135
 escopo de função 135
 escopo de protótipo de função 135
 escopo de um identificador 135
 especificadores de classe de armazenamento 132
 estouro de pilha 123
 etapa de recursão 138
 expressões de tipo misto 122
 fator de escala 125
 função 114
 função chamada 115
 função chamadora 115
 função recursiva 137
 funções definidas pelo programador 114
 invocar uma função 115
 last-in-first-out (LIFO) 123
 lista de parâmetros 118
 módulo 114
 números pseudoaleatórios 128
 ocultação de informações 135
 parâmetros 116
 pilha 123
 pilha de chamada de função 123
 pilha de execução do programa 123
 princípio do menor privilégio 135
 protótipo de função 118
 quadros de pilha 123
 randomização 128
 registros de ativação 123
 regras de promoção 122
 retorno de uma função 115
 reutilização do software 116
 semeia a função rand 128
 simulação 125
 static 132
 tipo-valor-retorno 118
 valor de deslocamento 129
 variáveis automáticas 133
 variáveis locais 116
 vinculação 132
 vinculação de um identificador 132

■ Exercícios de autorrevisão

5.1 Preencha os espaços nas seguintes sentenças:

- a)** Um módulo de programa em C é chamado de _____.
- b)** Uma função é invocada com um(a) _____.
- c)** Uma variável conhecida apenas dentro da função em que é definida é chamada de _____.
- d)** O comando _____ em uma função chamada é usado para passar o valor de uma expressão de volta à função chamadora.
- e)** A palavra-chave _____ é usada em um cabeçalho de função para indicar que uma função não retorna um valor ou para indicar que uma função não contém parâmetros.
- f)** O(A) _____ de um identificador é a parte do programa em que o identificador pode ser usado.
- g)** As três maneiras de retornar o controle de uma função chamada à chamadora são _____, _____ e _____.
- h)** Um(a) _____ permite que o compilador verifique o número, os tipos e a ordem dos argumentos passados a uma função.
- i)** A função _____ é usada para produzir números aleatórios.
- j)** A função _____ é usada para definir a semente do número aleatório para randomizar um programa.
- k)** Os especificadores de classe de armazenamento são _____, _____, _____ e _____.

- l) Supõe-se que as variáveis declaradas em um bloco ou na lista de parâmetros de uma função pertençam à classe de armazenamento _____, a menos que especificadas de outra maneira.
- m) O especificador de classe de armazenamento _____ é uma recomendação ao compilador para que armazene uma variável em um dos registradores do computador.
- n) Uma variável não `static` definida fora de qualquer bloco ou função é uma variável _____.
- o) Para que uma variável local em uma função retenha seu valor entre as chamadas a uma função, ela precisa ser declarada com o especificador de classe de armazenamento _____.
- p) Os quatro escopos possíveis de um identificador são _____, _____, _____ e _____.
- q) Uma função que chama a si mesma, direta ou indiretamente, é uma função _____.
- r) Uma função recursiva normalmente tem dois componentes: aquele que oferece um meio para a recursão terminar ao testar um caso _____ e um que expressa o problema como uma chamada recursiva para um problema ligeiramente mais simples que a chamada original.

5.2 Para o programa a seguir, indique o escopo (de função, de arquivo, de bloco ou de protótipo de função) de cada um dos seguintes elementos:

- a) A variável `x` em `main`.
- b) A variável `y` em `cube`.
- c) A função `cube`.
- d) A função `main`.
- e) O protótipo de função para `cube`.
- f) O identificador `y` no protótipo de função para `cube`.

```

1 #include <stdio.h>
2 int cube( int y );
3
4 int main( void )
5 {
6     int x;
7
8     for ( x = 1; x <= 10; x++ )
9         printf( "%d\n", cube( x ) );
10    return 0;
11 }
12
13 int cube( int y )
14 {
15     return y * y * y;
16 }
```

5.3 Escreva um programa que teste se os exemplos das chamadas de função da biblioteca matemática mostrados na Figura 5.2 realmente produzem os resultados indicados.

5.4 Indique o cabeçalho de função para as funções a seguir.

- a) Função `hypotenuse`, que utiliza dois argumentos de ponto flutuante e precisão dupla, `side1` e `side2`, e retorna um resultado de ponto flutuante e precisão dupla.
- b) Função `smallest`, que utiliza três inteiros, `x`, `y`, `z`, e retorna um inteiro.
- c) Função `instructions`, que não recebe argumento algum e não retorna um valor. [Nota: essas funções normalmente são usadas para exibir instruções a um usuário.]
- d) Função `intToFloat`, que utiliza um argumento inteiro, `number`, e retorna um resultado de ponto flutuante.

5.5 Dê o protótipo de função para cada um dos seguintes itens:

- a) A função descrita no Exercício 5.4(a).
- b) A função descrita no Exercício 5.4(b).
- c) A função descrita no Exercício 5.4(c).
- d) A função descrita no Exercício 5.4(d).

5.6 Escreva uma declaração para cada um dos seguintes itens:

- a) Inteiro `count` que deve ser mantido em um registrador. Inicialize `count` em 0.
- b) Variável de ponto flutuante `lastVal` que deve reter seu valor entre as chamadas para a função em que é definida.
- c) Inteiro externo `number` cujo escopo deve ser restrinado ao restante do arquivo em que está definido.

5.7 Encontre o erro em cada um destes segmentos de programa, e explique como ele pode ser corrigido (ver também o Exercício 5.46):

- a)

```

int g( void )
{
    printf( "Dentro da função g\n" );
    int h( void )
    {
        printf( "Dentro da função h\n" );
    }
}
```
- b)

```

int sum( int x, int y )
{
    int result;
    result = x + y;
}
```

c)

```
int sum( int n )
{
    if ( n == 0 ) {
        return 0;

    else {
        n + sum( n - 1 );
    }
}
```

d)

```
void f( float a );
{
```

```
float a;
printf( "%f", a );
}
```

e)

```
void product( void )
{
    int a, b, c, result;
    printf( "Digite três inteiros: " )
    scanf( "%d%d%d", &a, &b, &c );
    result = a * b * c;
    printf( "Resultado é %d", result );
    return result;
}
```

Respostas dos exercícios de autorrevisão

- 5.1** a) Função. b) Chamada de função. c) Variável local. d) return. e) void. f) Escopo. g) return; return expressão; ou encontrando a chave direita de fechamento de uma função. h) Protótipo de função. i) rand. j) srand. k) auto, register, extern, static. l) auto. m) register. n) Externa, global. o) static. p) Escopo de função, escopo de arquivo, escopo de bloco, escopo de protótipo de função. q) Recursiva. r) Básico.
- 5.2** a) Escopo de bloco. b) Escopo de bloco. c) Escopo de arquivo. d) Escopo de arquivo. e) Escopo de arquivo. f) Escopo de protótipo de função.
- 5.3** Veja a seguir. [Nota: na maior parte dos sistemas Linux, é preciso usar a opção -lm ao compilar esse programa.]

```
1 /* ex05_03.c */
2 /* Testando as funções da biblioteca matemática */
3 #include <stdio.h>
4 #include <math.h>
5
6 /* função main inicia a execução do programa */
7 int main( void )
8 {
9     /* calcula e informa a raiz quadrada */
10    printf( "sqrt(%.1f) = %.1f\n", 900.0, sqrt( 900.0 ) );
11    printf( "sqrt(%.1f) = %.1f\n", 9.0, sqrt( 9.0 ) );
12
13    /* calcula e informa a função exponencial e elevada a x */
```

```
14    printf( "exp(%.1f) = %f\n", 1.0, exp( 1.0 ) );
15    printf( "exp(%.1f) = %f\n", 2.0, exp( 2.0 ) );
16
17    /* calcula e informa o logaritmo (base e) */
18    printf( "log(%.1f) = %.1f\n", 2.718282, log( 2.718282 ) );
19    printf( "log(%.1f) = %.1f\n", 7.389056, log( 7.389056 ) );
20
21    /* calcula e apresenta o logaritmo (base 10) */
22    printf( "log10(%.1f) = %.1f\n", 1.0, log10( 1.0 ) );
23    printf( "log10(%.1f) = %.1f\n", 10.0, log10( 10.0 ) );
24    printf( "log10(%.1f) = %.1f\n", 100.0, log10( 100.0 ) );
25
26    /* calcula e apresenta o valor absoluto */
27    printf( "fabs(%.1f) = %.1f\n", 13.5, fabs( 13.5 ) );
28    printf( "fabs(%.1f) = %.1f\n", 0.0, fabs( 0.0 ) );
29    printf( "fabs(%.1f) = %.1f\n", -13.5, fabs( -13.5 ) );
30
31    /* calcula e informa ceil( x ) */
32    printf( "ceil(%.1f) = %.1f\n", 9.2, ceil( 9.2 ) );
33    printf( "ceil(%.1f) = %.1f\n", -9.8, ceil( -9.8 ) );
```

```

34
35  /* calcula e informa floor( x ) */
36  printf( "floor(%.1f) = %.1f\n", 9.2,
    floor( 9.2 ) );
37  printf( "floor(%.1f) = %.1f\n", -9.8,
    floor( -9.8 ) );
38
39  /* calcula e informa pow( x, y ) */
40  printf( "pow(%.1f, %.1f) = %.1f\n", 2.0,
    7.0, pow( 2.0, 7.0 ) );
41  printf( "pow(%.1f, %.1f) = %.1f\n", 9.0,
    0.5, pow( 9.0, 0.5 ) );
42
43  /* calcula e informa fmod( x, y ) */
44  printf( "fmod(%.3f/%.3f) = %.3f\n",
    13.675, 2.333,
45  fmod( 13.675, 2.333 ) );
46
47  /* calcula e informa sin( x ) */
48  printf( "sin(%.1f) = %.1f\n", 0.0, sin(
    0.0 ) );
49
50  /* calcula e informa cos( x ) */
51  printf( "cos(%.1f) = %.1f\n", 0.0, cos(
    0.0 ) );
52
53  /* calcula e informa tan( x ) */
54  printf( "tan(%.1f) = %.1f\n", 0.0, tan(
    0.0 ) );
55  return 0; /* indica conclusão bem-suce-
    dida */
56 } /* fim do main */

sqrt(900.0) = 30.0
sqrt(9.0) = 3.0
exp(1.0) = 2.718282
exp(2.0) = 7.389056
log(2.718282) = 1.0
log(7.389056) = 2.0
log10(1.0) = 0.0
log10(10.0) = 1.0
log10(100.0) = 2.0
fabs(13.5) = 13.5
fabs(0.0) = 0.0
fabs(-13.5) = 13.5
ceil(9.2) = 10.0
ceil(-9.8) = -9.0
floor(9.2) = 9.0
floor(-9.8) = -10.0

```

```

pow(2.0, 7.0) = 128.0
pow(9.0, 0.5) = 3.0
fmod(13.675/2.333) = 2.010
sin(0.0) = 0.0
cos(0.0) = 1.0
tan(0.0) = 0.0

```

5.4

- a) **double** hypotenuse(**double** side1, **double** side2)
- b) **int** smallest(**int** x, **int** y, **int** z)
- c) **void** instructions(**void**)
- d) **float** intToFloat(**int** number)

5.5

- a) **double** hypotenuse(**double** side1, **double** side2);
- b) **int** smallest(**int** x, **int** y, **int** z);
- c) **void** instructions(**void**);
- d) **float** intToFloat(**int** number);

5.6

- a) **register int** count = 0;
- b) **static float** lastVal;
- c) **static int** number;

[Nota: isso apareceria fora de qualquer definição de função.]

5.7

a) Erro: A função h é definida na função g. Correção: Mova a definição de h para fora da definição de g.

b) Erro: O corpo da função deveria retornar um inteiro, mas não o faz. Correção: Exclua a variável result e coloque o seguinte comando na função:

```
return x + y;
```

c) Erro: O resultado de $n + \text{sum}(n - 1)$ não é retornado; sum retorna um resultado impróprio. Correção: Reescreva o comando na cláusula else como

```
return n + sum( n - 1 );
```

d) Erro: Ponto e vírgula após o parêntese à direita que delimita a lista de parâmetros, e redefinição do parâmetro a na definição da função. Correção: Retire o ponto e vírgula após o parêntese à direita na lista de parâmetros e exclua a declaração float a; no corpo da função.

e) Erro: A função retorna um valor quando não deveria. Correção: Elimine o comando return.

Exercícios

5.8 Mostre o valor de x após a execução de cada uma das seguintes instruções:

- a) $x = \text{fabs}(7.5);$
- b) $x = \text{floor}(7.5);$
- c) $x = \text{fabs}(0.0);$
- d) $x = \text{ceil}(0.0);$
- e) $x = \text{fabs}(-6.4);$
- f) $x = \text{ceil}(-6.4);$
- g) $x = \text{ceil}(-\text{fabs}(-8 + \text{floor}(-5.5)));$

5.9 **Tarifa de estacionamento.** Um estacionamento cobra uma tarifa mínima de R\$ 2,00 por uma permanência de até três horas, e R\$ 0,50 adicionais por hora para cada hora, *ou parte dela*, por até três horas. A tarifa máxima para qualquer período de 24 horas é de R\$ 10,00. Suponha que nenhum carro estacione por mais de 24 horas de cada vez. Escreva um programa que calcule e imprima as tarifas de estacionamento para cada um dos três clientes que utilizaram esse estacionamento ontem. Você deverá informar as horas de permanência de cada cliente. Seu programa deverá imprimir os resultados em um formato tabular e deverá calcular e imprimir o total recebido ontem. O programa deverá usar a função `calcularTaxas` para determinar o valor devido por cliente. Sua saída deverá aparecer no seguinte formato:

Carro	Horas	Taxa
1	1,5	2,00
2	4,0	2,50
3	24,0	10,00
TOTAL	29,5	14,50

5.10 **Arredondando números.** Uma das aplicações da função `floor` é arredondar um valor para o inteiro mais próximo. A instrução

```
y = floor( x + .5 );
```

arredondará o número x para o inteiro mais próximo e atribuirá o resultado a y . Escreva um programa que leia vários números e use o comando anterior para arredondar cada um desses números para o inteiro mais próximo. Para cada número processado, imprima o número original e o número arredondado.

5.11 **Arredondando números.** A função `floor` pode ser usada para arredondar um número até uma casa decimal específica. A instrução

```
y = floor( x * 10 + 5 ) / 10;
```

arredonda x até a posição de décimos (a primeira posição à direita do ponto decimal). A instrução

```
y = floor( x * 100 + .5 ) / 100;
```

arredonda x até a posição de centésimos (a segunda posição à direita do ponto decimal). Escreva um programa que defina quatro funções que arredondem um número x de várias maneiras:

- a) `arredInteiro( número )`
- b) `arredDecimos( número )`
- c) `arredCentesimos( número )`
- d) `arredMilesimos( número )`

Para cada valor lido, seu programa deverá imprimir o valor original, o número arredondado até o próximo inteiro, o número arredondado até o próximo décimo, até o próximo centésimo e até o próximo milésimo.

5.12 Responda cada uma das seguintes perguntas.

- a) O que significa escolher números ‘aleatoriamente’?
- b) Por que a função `rand` é útil para simular jogos de azar?
- c) Por que você randomizaria um programa usando `srand`? Sob que circunstâncias é desejável não randomizar?
- d) Por que normalmente é necessário escalar e/ou deslocar os valores produzidos por `rand`?
- e) Por que a simulação computadorizada de situações do mundo real é uma técnica útil?

5.13 Escreva instruções que atribuam inteiros aleatórios à variável n nos seguintes intervalos:

- a) $1 \leq n \leq 2$
- b) $1 \leq n \leq 100$
- c) $0 \leq n \leq 9$
- d) $1000 \leq n \leq 1112$
- e) $-1 \leq n \leq 1$
- f) $-3 \leq n \leq 11$

5.14 Para cada um dos conjuntos de inteiros a seguir, escreva uma única instrução que imprima um número aleatoriamente a partir do conjunto.

- a) 2, 4, 6, 8, 10.
- b) 3, 5, 7, 9, 11.
- c) 6, 10, 14, 18, 22.

5.15 **Cálculos de hipotenusa.** Defina uma função chamada `hipotenusa` que calcule o tamanho da hipotenusa de um triângulo retângulo quando os outros dois lados são conhecidos. Use essa função em um programa para determinar o tamanho da hipotenusa de cada um dos triângulos da tabela a seguir. A função deverá usar dois argumentos do tipo `double` e retornar a hipotenusa como um `double`. Teste seu programa com os valores de lado especificados na Figura 5.18.

Triângulo	Lado 1	Lado 2
1	3,0	4,0
2	5,0	12,0
3	8,0	15,0

Figura 5.18 ■ Valores de lado do triângulo para o Exercício 5.15.

5.16 Exponenciação Escreva uma função `potenciaInt(base, expoente)` que retorne o valor de $\text{base}^{\text{expoente}}$

Por exemplo, $\text{potenciaInt}(3, 4) = 3 * 3 * 3 * 3$. Suponha que `expoente` seja um inteiro positivo, diferente de zero, e `base` seja um inteiro. A função `potenciaInt` deve usar `for` para controlar o cálculo. Não use funções da biblioteca matemática.

5.17 Múltiplos. Escreva uma função `multiple` que determine para um par de inteiros, se o segundo inteiro é um múltiplo do primeiro. A função deve apanhar dois argumentos inteiros e retornar 1 (verdadeiro), se o segundo for um múltiplo do primeiro, e 0 (falso), caso contrário. Use essa função em um programa que inclua uma série de pares de inteiros.

5.18 Par ou ímpar. Escreva um programa que inclua uma série de inteiros e os passe um de cada vez à função `even`, que usa o operador de módulo para determinar se um inteiro é par. A função deverá usar um argumento inteiro e retornar 1 se o inteiro for par, e retornar 0 se o inteiro for ímpar.

5.19 Exibindo um quadrado de asteriscos. Escreva uma função que apresente um quadrado sólido de asteriscos cujo lado é especificado no parâmetro inteiro `side`. Por exemplo, se `side` é 4, a função exibe:

```
****
****
****
****
```

5.20 Exibindo o quadrado de um caractere. Modifique a função criada no Exercício 5.19 para formar um quadrado a partir de qualquer caractere contido no parâmetro de caractere `fillCharacter`. Assim, se `side` é 5 e `fillCharacter` é '#', então essa função deverá imprimir:

```
#####
#####
#####
#####
#####
```

5.21 Projeto: desenhando formas com caracteres.

Use técnicas semelhantes às que foram desenvolvidas nos exercícios 5.19 a 5.20 para produzir um programa que represente graficamente uma grande variedade de formas.

5.22 Separando dígitos. Escreva segmentos de programa que realizem, cada um, o seguinte:

- O cálculo da parte inteira do quociente quando o inteiro `a` é dividido pelo inteiro `b`.
- O cálculo do módulo inteiro quando o inteiro `a` é dividido pelo inteiro `b`.
- Use as partes do programa desenvolvidas em a) e b) para escrever uma função que peça um inteiro entre 1 e 32767 e o imprima como uma série de dígitos, com dois espaços entre cada dígito. Por exemplo, o inteiro 4562 deverá ser impresso como:

```
4 5 6 2
```

5.23 Tempo em segundos. Escreva uma função que considere a hora como três argumentos inteiros (horas, minutos e segundos) e retorne o número de segundos desde a última vez em que o relógio 'bateu 12 horas'. Use essa função para calcular a quantidade de tempo em segundos entre duas horas, ambas dentro de um ciclo de 12 horas do relógio.

5.24 Conversões de temperatura. Implemente as seguintes funções com inteiros:

- Função `celsius` retorna o equivalente Celsius de uma temperatura em Fahrenheit.
- Função `fahrenheit` retorna o equivalente Fahrenheit de uma temperatura em Celsius.
- Use essas funções para escrever um programa que imprima gráficos mostrando os equivalentes em Fahrenheit de todas as temperaturas Celsius de 0 a 100 graus, e os equivalentes em Celsius de todas as temperaturas fahrenheit de 32 a 212 graus. Imprima as saídas em um formato tabular que reduza o número de linhas de saída enquanto continua sendo legível.

5.25 Achar o mínimo. Escreva uma função que retorne o menor de três números em ponto flutuante.

5.26 Números perfeitos. Um número inteiro é considerado um *número perfeito* se a soma de seus fatores, incluindo 1 (mas não o próprio número) for igual ao próprio número. Por exemplo, 6 é um número perfeito porque $6 = 1 + 2 + 3$. Escreva uma função `perfect` que determine se o parâmetro `number` é um número perfeito. Use essa função em um programa que determine e imprima todos os números perfeitos entre 1 e 1000. Imprima os fatores de cada número perfeito para confirmar se o número é realmente perfeito. Desafie o poder de seu computador testando números muito maiores que 1000.

5.27 Números primos. Um inteiro é considerado *primo* se for divisível apenas por 1 e por ele mesmo. Por exemplo, 2, 3, 5 e 7 são primos, mas 4, 6, 8 e 9 não são.

- Escreva uma função que determine se um número é primo.
- Use essa função em um programa que determine e imprima todos os números primos entre 1 e 10000. Quantos desses 10000 números você realmente precisa testar antes de ter certeza de que encontrou todos os primos?
- Inicialmente, você poderia pensar que $n/2$ é o limite superior dentro do qual deveria testar para ver se um número é primo, mas você só precisa ir até a raiz quadrada de n . Por quê? Reescreva o programa e execute-o das duas maneiras. Estime a melhoria do desempenho.

5.28 Invertendo dígitos. Escreva uma função que leia um valor inteiro e retorne o número com seus dígitos invertidos. Por exemplo, dado o número 7631, a função deverá retornar 1367.

5.29 Máximo divisor comum. O *máximo divisor comum* (MDC) de dois inteiros é o maior inteiro que divide cada um dos dois números sem deixar resto. Escreva a função `mdc` que retorna o máximo divisor comum de dois inteiros.

5.30 Escreva uma função `qualityPoints` que peça a média de um estudante e retorne 4 se a média for 90-100, 3 se a média for 80-89, 2 se a média for 70-79, 1 se a média for 60-69 e 0 se a média for menor que 60.

5.31 Jogando uma moeda. Escreva um programa que simule o lançamento de uma moeda. Para cada lançamento da moeda, o programa deverá imprimir Cara ou Coroa. Deixe o programa jogar a moeda 100 vezes e conte o número de vezes que cada lado da moeda aparece. Imprima os resultados. O programa deverá chamar uma função `flip` separada, que não utilize argumentos e retorne 0 para cara e 1 para coroa. [Nota: se o programa realisticamente simula o lançamento de uma moeda, então cada lado da moeda deve aparecer aproximadamente em metade do tempo para um total de aproximadamente 50 caras e 50 coroas.]

5.32 Descubra o número. Escreva um programa em C que contenha o jogo 'descubra o número', da seguinte forma: seu programa escolhe o número a ser descoberto, selecionando um inteiro aleatório na faixa de 1 a 1000. O programa então exibe:

Eu tenho um número entre 1 e 1000.
 Você consegue descobrir qual é?
 Digite sua primeira tentativa.

O jogador, então, digita uma primeira tentativa. O programa responde com uma das seguintes sentenças:

```
1 Excelente! Você descobriu o número!
   Gostaria de jogar novamente (s ou n)?
2 Muito baixo. Tente novamente.
3 Muito alto. Tente novamente.
```

Se a escolha do jogador estiver incorreta, seu programa deverá realizar um loop até que o jogador finalmente acerte o número. Seu programa deverá continuar informando ao jogador Muito alto ou Muito baixo para ajudá-lo a 'mirar' na resposta correta. [Nota: a técnica de pesquisa empregada nesse problema é chamada de pesquisa binária. Falaremos mais sobre isso no próximo problema.]

5.33 Modificação de 'Descubra o número'. Modifique o programa do Exercício 5.32 para contar o número de tentativas que o jogador fez. Se o número for 10 ou menos, imprima Ou você conhece o segredo ou teve sorte! Se o jogador acertar o número em 10 tentativas, então imprima Ahah! Você conhece o segredo! Se o jogador conseguir em mais de 10 tentativas, então imprima Você deveria se sair melhor! Por que não deveria passar de 10 tentativas? Bem, com cada 'escolha boa', o jogador deveria poder eliminar metade dos números. Agora, mostre por que qualquer número de 1 a 1000 pode ser descoberto em 10 ou menos tentativas.

5.34 Exponenciação recursiva. Escreva uma função recursiva `power( base, expoente )` que, quando chamada, retorna

`baseexpoente`

Por exemplo, `power( 3, 4 ) = 3 * 3 * 3 * 3`. Suponha que `expoente` seja um inteiro maior ou igual a 1. Dica: a etapa de recursão usaria o relacionamento

`baseexpoente = base * baseexpoente-1`

e a condição de término ocorre quando `expoente` é igual a 1, pois

`base1 = base`

5.35 Fibonacci. A série de Fibonacci

0, 1, 1, 2, 3, 5, 8, 13, 21, ...

começa com os termos 0 e 1, e tem a propriedade de estabelecer que o termo seguinte é a soma dos dois termos anteriores. a) Escreva uma função *não recursiva* fibonacci(*n*) que calcule o *n-ésimo* número de Fibonacci. b) Determine o maior número de Fibonacci que pode ser impresso no seu sistema. Modifique o programa da parte a) para usar `double` em vez de `int` para calcular e retornar números de Fibonacci. Faça com que o programa se repita até que falhe devido a um valor excessivamente alto.

5.36 Torres de Hanói. Todo cientista da computação iniciante precisa lutar contra certos problemas clássicos, e as Torres de Hanói (ver Figura 5.19) é um dos problemas mais famosos. A lenda diz que, em um templo no Extremo Oriente, sacerdotes estão tentando mover uma pilha de discos de um pino para outro. A pilha inicial tinha 64 discos dispostos em um pino e arrumados de baixo para cima por tamanho decrescente. Os sacerdotes estão tentando mover a pilha desse pino para um segundo pino sob as restrições de que apenas um disco é movido de cada vez, e em momento algum um disco maior pode ser colocado sobre um disco menor. Um terceiro pino está disponível para apoiar os discos temporariamente. Supostamente, o mundo terminará quando os sacerdotes completarem sua tarefa, de modo que não temos muito incentivo para facilitar seus esforços.

Vamos supor que os sacerdotes estejam tentando mover os discos do pino 1 ao pino 3. Queremos desenvolver um algoritmo que imprimirá a sequência exata de transferências de pino, disco a disco.

Se tivéssemos que resolver esse problema com a ajuda de métodos convencionais, ficaríamos presos na administração dos discos rapidamente. Em vez disso, se atacarmos o problema com a recursão em mente, ele imediatamente se tornará mais resolúvel. Mover *n* discos pode

ser visto em termos de mover apenas $n - 1$ discos (e daí a recursão) da seguinte forma:

- a) Mova $n - 1$ discos do pino 1 para o pino 2, usando o pino 3 como área de manutenção temporária.
- b) Mova o último disco (o maior) do pino 1 para o pino 3.
- c) Mova os $n - 1$ discos do pino 2 para o pino 3, usando o pino 1 como área de manutenção temporária.

O processo termina quando a última tarefa envolver mover $n = 1$ disco, ou seja, o caso básico. Isso é feito movendo o disco de modo trivial, sem a necessidade de uma área de manutenção temporária.

Escreva um programa que resolva o problema das Torres de Hanói. Use uma função recursiva com quatro parâmetros:

- a) O número de discos a serem movidos.
- b) O pino no qual esses discos estão empilhados inicialmente.
- c) O pino para o qual a pilha de discos deve ser movida.
- d) O pino a ser usado como área de manutenção temporária.

Seu programa deverá imprimir as instruções exatas que ele usará para mover os discos do pino inicial para o pino de destino. Por exemplo, para mover uma pilha de três discos do pino 1 ao pino 3, seu programa deverá imprimir a seguinte série de movimentos:

1 → 3 (Isso significa mover um disco do pino 1 ao pino 3.)

1 → 2

3 → 2

1 → 3

2 → 1

2 → 3

1 → 3

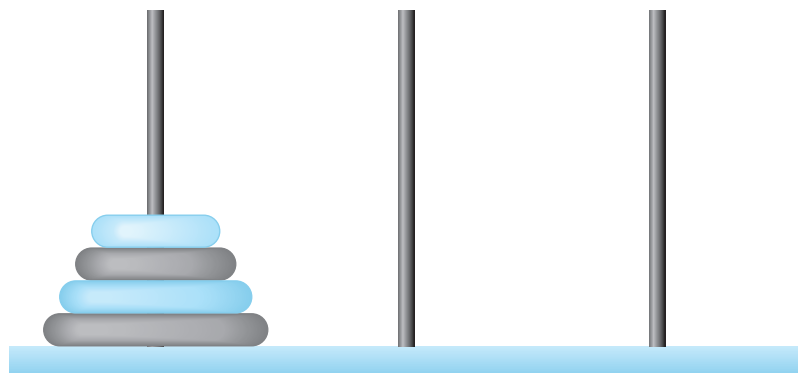


Figura 5.19 ■ Torres de Hanói para o caso dos quatro discos.

5.37 Torres de Hanói: solução iterativa. Qualquer programa que possa ser implementado recursivamente pode ser implementado iterativamente, embora, às vezes, com muito mais dificuldade e muito menos clareza. Tente escrever uma versão iterativa das Torres de Hanói. Se obtiver sucesso, compare sua versão iterativa com a versão recursiva desenvolvida por você no Exercício 5.36. Investigue as questões de desempenho, clareza e sua capacidade de demonstrar a exatidão dos programas.

5.38 Visualizando a recursão. É interessante observar a recursão ‘em ação’. Modifique a função fatorial da Figura 5.14 para imprimir sua variável local e o parâmetro de chamada recursiva. Para cada chamada recursiva, apresente as saídas em uma linha separada e acrescente um nível de recuo. Faça o máximo para tornar as saídas claras, interessantes e significativas. Seu objetivo aqui é projetar e implementar um formato de saída que ajude uma pessoa a entender melhor a recursão. Você pode querer acrescentar essas capacidades de exibição a muitos outros exemplos e exercícios de recursão no decorrer do livro.

5.39 Máximo divisor comum recursivo. O máximo divisor comum dos inteiros x e y é o maior inteiro que divide x e y sem gerar resto. Escreva uma função recursiva `mdc` que retorne o máximo divisor comum de x e y . O `mdc` de x e y é definido recursivamente da seguinte forma: se y é igual a 0, então `mdc(x, y)` é x ; caso contrário, `mdc(x, y)` é `mdc(y, x % y)`, onde `%` é o operador de módulo (ou resto da divisão).

5.40 main recursiva. `main` pode ser chamada recursivamente? Escreva um programa que contenha uma função `main`. Inclua a variável local `static count` inicializada como 1. Pós-incrementa e imprima o valor de `count` cada vez que `main` for chamada. Execute seu programa. O que acontece?

5.41 Distância entre pontos. Escreva a função `distance` que calcula a distância entre dois pontos (x_1, y_1) e (x_2, y_2) . Todos os números e valores de retorno devem ser do tipo `double`.

5.42 O que o programa a seguir faz?

```
1 #include <stdio.h>
2
3 /* função main inicia a execução do pro-
   programa */
4 int main( void )
5 {
6     int c; /* variável para manter entra-
       da de caractere pelo usuário */
7
8     if ( ( c = getchar() ) != EOF ) {
9         main();
10        printf( "%c", c );
11    } /* fim do if */
12
13    return 0; /* indica conclusão bem-
       sucedida */
14 } /* fim do main */
```

5.43 O que o programa a seguir faz?

```
1 #include <stdio.h>
2
3 int mystery( int a, int b ); /* protóti-
   po de função */
4
5 /* função main inicia a execução do pro-
   grama */
6 int main( void )
7 {
8     int x; /* primeiro inteiro */
9     int y; /* segundo inteiro */
10
11    printf( "Digite dois inteiros: " );
12    scanf( "%d%d", &x, &y );
13
14    printf( "O resultado é %d\n", mys-
       tery( x, y ) );
15    return 0; /* indica conclusão bem-
       sucedida */
16 } /* fim do main */
17
18 /* Parâmetro b deve ser um inteiro po-
   sitivo,
19    para impedir recursão infinita */
20 int mystery( int a, int b )
21 {
22     /* caso básico */
23     if ( b == 1 ) {
24         return a;
25     } /* fim do if */
26     else { /* etapa recursiva */
27         return a + mystery( a, b - 1 );
28     } /* fim do else */
29 } /* fim da função mystery */
```

5.44 Depois de determinar o que o programa do Exercício 5.43 faz, modifique-o para que funcione devidamente depois de removida a restrição que estabelece que o segundo argumento não pode ser negativo.

5.45 Testando funções da biblioteca matemática. Escreva um programa que teste o maior número possível de funções de biblioteca da Figura 5.2. Exercite cada uma dessas funções fazendo com que o programa imprima tabelas de valores de retorno para uma diversidade de valores de argumento.

5.46 Encontre o erro em cada um dos segmentos de programa a seguir e explique como corrigi-los:

- a) `double cube( float ); /* protótipo de função */`
`cube( float number ) /* function definition */`
`{`
`return number * number * number;`
`}`
- b) `register auto int x = 7;`
- c) `int randomNumber = srand();`
- d) `double y = 123.45678;`
`int x;`
`x = y;`
`printf( "%f\n", (double) x );`
- e) `double square( double number )`
`{`
`double number;`
`return number * number;`
`}`
- f) `int sum( int n )`
`{`
`if ( n == 0 ) {`
`return 0;`
`}`
`else {`
`return n + sum( n );`
`}`
`}`

5.47 Modificação do jogo 'craps'. Modifique o programa do jogo 'craps' da Figura 5.10 para que ele permita apostas. Inicialize a variável `saldoBanca` como 1.000 reais. Peça ao jogador que informe uma aposta. Use um loop `while` para verificar se a aposta é menor ou igual a `saldoBanca` e, se não for, peça ao usuário que informe aposta novamente até uma aposta válida ser informada. Depois que a aposta correta tiver sido informada, execute um jogo de craps. Se o jogador vencer, aumente `saldoBanca` em aposta e imprima o novo `saldoBanca`. Se o jogador perder, diminua `saldoBanca` em aposta, imprima o novo `saldoBanca`, verifique se `saldoBanca` chegou a zero e, nesse caso, imprima a mensagem: "Sinto muito. Você está quebrado!" Conforme o jogo progride, imprima diversas mensagens com o intuito de criar uma "conversa", como "Ei, assim você quebra!" ou "Vamos lá, continue tentando!" ou "Você está ganhando. Agora é hora de aproveitar suas fichas!"

5.48 Projeto de pesquisa: aperfeiçoando a implementação de Fibonacci recursiva. Na Seção 5.15, o algoritmo recursivo que usamos para calcular números de Fibonacci foi intuitivamente atraente. Porém, lembre-se de que o algoritmo resultou na explosão exponencial das chamadas de função recursivas. Pesquise a implementação de Fibonacci recursiva on-line. Estude as diversas técnicas, incluindo a versão iterativa do Exercício 5.35 e as versões que usam apenas a chamada 'recursão de cauda' (*tail recursion*). Discuta os méritos relativos de cada uma.

Fazendo a diferença

5.49 Teste sobre o aquecimento global. A controvertida questão do aquecimento global tem sido bastante alardeada pelo filme *An Inconvenient Truth* (*Uma verdade inconveniente*), estrelado pelo ex-vice presidente Al Gore. Gore e um grupo de cientistas das Nações Unidas, o Intergovernmental Panel on Climate Change, compartilharam o Prêmio Nobel da Paz em 2007 em reconhecimento por 'seus esforços para construir e disseminar um conhecimento mais amplo sobre a mudança do clima ocasionada pelo homem'. Pesquise os dois lados da questão do aquecimento global on-line (você pode procurar frases como 'global warming skeptics' — céticos do aquecimento global). Crie um teste de múltipla escolha com cinco perguntas sobre o aquecimento global, com cada pergunta tendo quatro respostas possíveis (numeradas de 1 a 4). Seja objetivo e tente representar de forma imparcial os dois lados da questão. Em seguida, escreva uma aplicação que administre o teste, calcule o número de respostas corretas (de zero a cinco) e

retorne uma mensagem ao usuário. Se o usuário responder corretamente as cinco perguntas, imprima 'Excelente'; se acertar quatro, imprima 'Muito bom'; se forem três ou menos, imprima 'Hora de aumentar seus conhecimentos sobre aquecimento global', e inclua uma lista de alguns dos sites em que encontrou as informações usadas.

Instrução auxiliada por computador

Com o preço dos computadores em queda, vai-se tornando viável a todo estudante, independentemente de sua situação econômica, ter um computador e usá-lo na escola. Isso cria possibilidades interessantes de melhora da experiência educacional de todos os alunos no mundo inteiro, conforme sugerimos nos próximos cinco exercícios. [Nota: verifique iniciativas como One Laptop Per Child Project (<www.laptop.org>). Além disso, pesquise laptops 'verdes' — quais são algumas das características-chave desses dispositivos que estão

‘esverdeando’? Examine a Electronic Product Environmental Assessment Tool (<www.epeat.net>), que poderá ajudá-lo a avaliar o ‘grau de verde’ dos desktops, notebooks e monitores, ajudando-o a decidir quais produtos comprar.]

5.50 Instrução auxiliada por computador. O uso de computadores na educação é conhecido como *Instrução Auxiliada por Computador (IAC)*. Escreva um programa que ajude um aluno do ensino fundamental a aprender a tabuada. Use a função `rand` para produzir dois inteiros positivos de um dígito. O programa deverá então fazer uma pergunta ao usuário, tal como

Quanto é 6 vezes 7?

O aluno, então, entra com a resposta. Em seguida, o programa verifica a resposta do aluno. Se estiver correta, mostre a mensagem ‘Muito bom!’ e faça outra pergunta de tabuada. Se a resposta estiver errada, mostre a mensagem ‘Não. Tente novamente.’ e deixe que o aluno tente responder à mesma pergunta repetidamente, até que ele finalmente acerte a resposta. Uma função separada deverá ser usada para gerar cada nova pergunta. Essa função deverá ser chamada uma vez, quando a aplicação iniciar a execução, e toda vez que o usuário responder à pergunta corretamente.

5.51 Instrução auxiliada por computador: reduzindo a fadiga do estudante. Um problema nos ambientes de IAC é a fadiga do estudante. Modifique o programa do Exercício 5.50 de modo que vários comentários sejam exibidos para cada resposta da seguinte forma:

Possíveis mensagens para uma resposta correta:

Muito bom!
Excelente!
Bom trabalho!
Continue assim!

Possíveis mensagens para uma resposta incorreta:

Não. Tente novamente.
Errado. Tente mais uma vez.

Não desista!

Não. Continue tentando.

Use a geração de número aleatório para escolher um número de 1 a 4, que será usado para selecionar uma das respostas apropriadas para cada resposta correta ou incorreta. Use uma estrutura `switch` para emitir as respostas.

5.52 Instrução auxiliada por computador: monitorando o desempenho do estudante. Sistemas mais sofisticados de instrução auxiliada por computador monitoram o desempenho do estudante por um período. A decisão de iniciar um novo tópico normalmente é baseada no sucesso do estudante com os tópicos anteriores. Modifique o programa do Exercício 5.51 para contar o número de respostas corretas e incorretas digitadas pelo estudante. Depois que o estudante digitar 10 respostas, seu programa deverá calcular sua porcentagem correta. Se a porcentagem for menor que 75 por cento, mostre a mensagem ‘Por favor, peça ajuda a seu professor.’, depois reinicie o programa para que outro estudante possa responder. Se a porcentagem for 75 por cento ou maior, mostre a mensagem ‘Parabéns, você está pronto para o próximo nível!’, e depois reinicie o programa para que outro estudante possa responder.

5.53 Instrução auxiliada por computador: níveis de dificuldade. Os exercícios 5.50 a 5.52 desenvolveram um programa de instrução auxiliada por computador para ajudar a ensinar multiplicação a um aluno do ensino fundamental. Modifique o programa de modo a permitir que o usuário entre com um nível de dificuldade. Em um nível de dificuldade 1, o programa deverá usar apenas números de um dígito nos problemas; em um nível de dificuldade 2, os números podem ter dois dígitos e assim por diante.

5.54 Instrução auxiliada por computador: variando os tipos de problemas. Modifique o programa do Exercício 5.53 de modo a permitir que o usuário escolha um tipo de problema aritmético para estudar. Uma opção 1 significa problemas de adição, 2 significa problemas de subtração, 3 significa problemas de multiplicação e 4 significa uma mistura aleatória de todos esses tipos.